

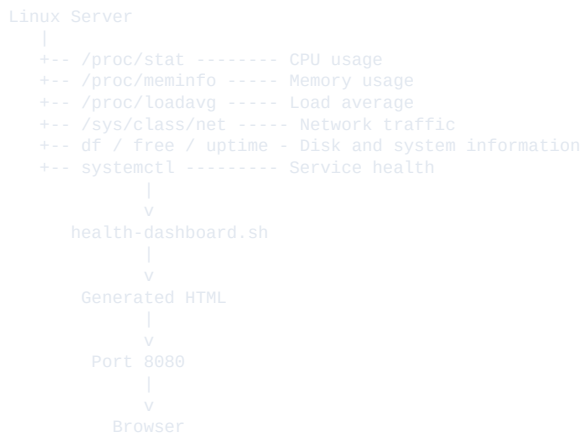
Linux Health Monitoring Dashboard

Complete Bash/Shell Script + Installation & Deployment Guide

Project goal: Build a Linux server health dashboard using Bash, Linux /proc and /sys filesystems, standard Linux commands, systemd service checks, and a lightweight HTTP server.

1. Project Architecture

The monitoring logic is written in Bash. The script reads Linux system information, generates an HTML dashboard, and serves it on port 8080. The browser automatically refreshes the page every 5 seconds.



2. Dashboard Features

- CPU utilization and CPU core count
- Memory utilization, total, used, and available memory
- Root filesystem utilization, total, used, and free space
- Load average and process count
- Hostname, operating system, kernel, and architecture
- Network interface, received bytes, and transmitted bytes
- SSH, Nginx, Docker, and Jenkins service status
- Responsive HTML dashboard
- Automatic refresh every 5 seconds

3. Requirements

Recommended: Amazon Linux 2023, Ubuntu, Debian, or another systemd-based Linux distribution. The script must run with root privileges because it checks system services and reads system information.

Required commands: bash, awk, grep, df, free, uptime, hostname, uname, systemctl, nproc. Python 3 or BusyBox is used only to serve the generated HTML page; the monitoring/dashboard logic remains Bash.

4. Installation

Create the project directory:

```
mkdir -p linux-health-dashboard
cd linux-health-dashboard
```

Create the script:

```
vi health-dashboard.sh
# or
nano health-dashboard.sh
```

Paste the complete script from Section 7, save it, and make it executable:

```
chmod +x health-dashboard.sh
sudo ./health-dashboard.sh
```

5. EC2 Deployment

For an AWS EC2 server, add an inbound Security Group rule for TCP port 8080. Use your EC2 public IPv4 address in the browser.

Setting	Value
Type	Custom TCP
Protocol	TCP
Port	8080
Source	Your IP/32 recommended; 0.0.0.0/0 only for temporary testing

<http://YOUR-EC2-PUBLIC-IP:8080>

6. Execution Flow

1. The script verifies root access.
2. It checks required Linux commands.
3. It creates /tmp/linux-health-dashboard.
4. It collects metrics from /proc, /sys, df, free, uptime, and systemctl.
5. It generates index.html.
6. Python 3 or BusyBox serves the HTML on port 8080.
7. The browser refreshes every 5 seconds.

7. Complete Script

The following is the complete validated Bash script. Save it as **health-dashboard.sh**.

```
#!/bin/bash
# =====
# Linux Health Monitoring Dashboard
# Bash-based monitoring dashboard
# =====

PORT=8080
HOST="0.0.0.0"
REFRESH=5

if [ "$EUID" -ne 0 ]; then
    echo "Please run this script as root."
    echo "Example: sudo bash health-dashboard.sh"
    exit 1
fi

COMMANDS=("awk" "sed" "grep" "df" "free" "uptime" "hostname" "uname" "systemctl")

for command in "${COMMANDS[@]}; do
    if ! command -v "$command" >/dev/null 2>&1; then
        echo "ERROR: $command is not installed."
        exit 1
    fi
done

WORK_DIR="/tmp/linux-health-dashboard"
mkdir -p "$WORK_DIR"
HTML_FILE="$WORK_DIR/index.html"

get_cpu_usage() {
    read cpu user nice system idle iowait irq softirq steal guest guest_nice < /proc/stat
    total1=$((user + nice + system + idle + iowait + irq + softirq + steal))
    idle1=$((idle + iowait))
    sleep 1
    read cpu user nice system idle iowait irq softirq steal guest guest_nice < /proc/stat
    total2=$((user + nice + system + idle + iowait + irq + softirq + steal))
    idle2=$((idle + iowait))
    total_diff=$((total2 - total1))
    idle_diff=$((idle2 - idle1))
    if [ "$total_diff" -eq 0 ]; then
```

```

        echo "0"
    else
        echo $((100 * (total_diff - idle_diff) / total_diff))
    fi
}

get_memory_usage() {
    total=$(awk '/MemTotal/ {print $2}' /proc/meminfo)
    available=$(awk '/MemAvailable/ {print $2}' /proc/meminfo)
    used=$((total - available))
    if [ "$total" -gt 0 ]; then
        echo $((used * 100 / total))
    else
        echo "0"
    fi
}

get_disk_usage() { df -P / | awk 'NR==2 {gsub("%","", $5); print $5}'; }
get_disk_free() { df -h / | awk 'NR==2 {print $4}'; }
get_disk_total() { df -h / | awk 'NR==2 {print $2}'; }
get_disk_used() { df -h / | awk 'NR==2 {print $3}'; }
get_memory_total() { free -h | awk '/Mem:/ {print $2}'; }
get_memory_used() { free -h | awk '/Mem:/ {print $3}'; }
get_memory_available() { free -h | awk '/Mem:/ {print $7}'; }
get_load_average() { awk '{print $1}' /proc/loadavg; }
get_process_count() { ls -d /proc/[0-9]* 2>/dev/null | wc -l; }
get_cpu_cores() { nproc; }
get_uptime() { uptime -p; }
get_hostname() { hostname; }

get_os() {
    if [ -f /etc/os-release ]; then
        . /etc/os-release
        echo "$PRETTY_NAME"
    else
        echo "Unknown Linux"
    fi
}

get_kernel() { uname -r; }
get_architecture() { uname -m; }

get_interface() {
    if command -v ip >/dev/null 2>&1; then
        ip route 2>/dev/null | awk '/default/ {print $5; exit}'
    else
        echo "N/A"
    fi
}

get_rx() {
    interface=$(get_interface)
    if [ -n "$interface" ] && [ -f "/sys/class/net/$interface/statistics/rx_bytes" ]; then
        bytes=$(cat "/sys/class/net/$interface/statistics/rx_bytes")
        if [ "$bytes" -ge 1073741824 ]; then
            awk "BEGIN {printf \"%.2f GB\", $bytes/1073741824}"
        elif [ "$bytes" -ge 1048576 ]; then
            awk "BEGIN {printf \"%.2f MB\", $bytes/1048576}"
        elif [ "$bytes" -ge 1024 ]; then
            awk "BEGIN {printf \"%.2f KB\", $bytes/1024}"
        else
            echo "${bytes} B"
        fi
    else
        echo "N/A"
    fi
}

get_tx() {
    interface=$(get_interface)
    if [ -n "$interface" ] && [ -f "/sys/class/net/$interface/statistics/tx_bytes" ]; then
        bytes=$(cat "/sys/class/net/$interface/statistics/tx_bytes")
        if [ "$bytes" -ge 1073741824 ]; then
            awk "BEGIN {printf \"%.2f GB\", $bytes/1073741824}"
        elif [ "$bytes" -ge 1048576 ]; then
            awk "BEGIN {printf \"%.2f MB\", $bytes/1048576}"
        elif [ "$bytes" -ge 1024 ]; then
            awk "BEGIN {printf \"%.2f KB\", $bytes/1024}"
        else
            echo "${bytes} B"
        fi
    else
        echo "N/A"
    fi
}

service_status() {
    service="$1"
    if systemctl list-unit-files 2>/dev/null | grep -q "^${service}\.service"; then
        if systemctl is-active --quiet "$service"; then

```

```

        echo "RUNNING"
    else
        echo "STOPPED"
    fi
else
    echo "NOT INSTALLED"
fi
}

status_class() {
    case "$1" in
        RUNNING) echo "running" ;;
        STOPPED) echo "stopped" ;;
        *) echo "unknown" ;;
    esac
}

generate_html() {
    CPU=$(get_cpu_usage)
    MEMORY=$(get_memory_usage)

    DISK=$(get_disk_usage)
    LOAD=$(get_load_average)
    PROCESSES=$(get_process_count)
    CORES=$(get_cpu_cores)
    HOSTNAME=$(get_hostname)
    OS=$(get_os)
    KERNEL=$(get_kernel)
    ARCH=$(get_architecture)
    UPTIME=$(get_uptime)
    MEMORY_TOTAL=$(get_memory_total)
    MEMORY_USED=$(get_memory_used)
    MEMORY_AVAILABLE=$(get_memory_available)
    DISK_TOTAL=$(get_disk_total)
    DISK_USED=$(get_disk_used)
    DISK_FREE=$(get_disk_free)
    INTERFACE=$(get_interface)
    RX=$(get_rx)
    TX=$(get_tx)

    SSH_STATUS=$(service_status sshd)
    NGINX_STATUS=$(service_status nginx)
    DOCKER_STATUS=$(service_status docker)
    JENKINS_STATUS=$(service_status jenkins)

    SSH_CLASS=$(status_class "$SSH_STATUS")
    NGINX_CLASS=$(status_class "$NGINX_STATUS")
    DOCKER_CLASS=$(status_class "$DOCKER_STATUS")
    JENKINS_CLASS=$(status_class "$JENKINS_STATUS")

    cat > "$HTML_FILE" <<EOF
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="refresh" content="$REFRESH">
<title>Linux Health Dashboard</title>
<style>
* { box-sizing: border-box; }
body {
    margin: 0;
    font-family: Arial, Helvetica, sans-serif;
    background: #0b1120;
    color: #f8fafc;
}
.container { max-width: 1400px; margin: auto; padding: 30px; }
.header {
    display: flex; justify-content: space-between; align-items: center;
    margin-bottom: 30px;
}
.title h1 { margin: 0; font-size: 32px; }
.title p { color: #94a3b8; margin-top: 8px; }
.status {
    background: #052e16; color: #4ade80; padding: 10px 18px;
    border-radius: 30px; font-weight: bold;
}
.grid {
    display: grid; grid-template-columns: repeat(4, 1fr); gap: 20px;
}
.card {
    background: #111827; border: 1px solid #1f2937; border-radius: 16px;
    padding: 22px; box-shadow: 0 10px 30px rgba(0,0,0,.2);
}
.card h3 {
    margin: 0; color: #94a3b8; font-size: 14px;
    text-transform: uppercase; letter-spacing: 1px;
}
.value { font-size: 34px; font-weight: bold; margin-top: 15px; }
.progress {
    height: 9px; background: #1f2937; border-radius: 10px;

```

```

        margin-top: 15px; overflow: hidden;
    }
    .progress-bar { height: 100%; background: #22c55e; }
    .section { margin-top: 25px; }
    .section-title { font-size: 20px; margin-bottom: 15px; }

    .info-grid {
        display: grid; grid-template-columns: repeat(3, 1fr); gap: 15px;
    }
    .info {
        background: #0f172a; border: 1px solid #1e293b;
        padding: 18px; border-radius: 12px;
    }
    .info span {
        display: block; color: #64748b; font-size: 13px; margin-bottom: 6px;
    }
    .services {
        display: grid; grid-template-columns: repeat(4, 1fr); gap: 15px;
    }
    .service {
        background: #0f172a; border-radius: 12px; padding: 18px;
        border: 1px solid #1e293b;
    }
    .service-name { font-weight: bold; margin-bottom: 12px; }
    .badge {
        display: inline-block; padding: 6px 10px; border-radius: 20px;
        font-size: 12px; font-weight: bold;
    }
    .running { background: #052e16; color: #4ade80; }
    .stopped { background: #450a0a; color: #f87171; }
    .unknown { background: #422006; color: #fbbf24; }
    .footer {
        text-align: center; color: #64748b; margin-top: 30px; padding: 20px;
    }
    @media(max-width: 1000px) {
        .grid { grid-template-columns: repeat(2, 1fr); }
        .services { grid-template-columns: repeat(2, 1fr); }
        .info-grid { grid-template-columns: repeat(2, 1fr); }
    }
    @media(max-width: 600px) {
        .container { padding: 15px; }
        .header { display: block; }
        .status { display: inline-block; margin-top: 15px; }
        .grid, .services, .info-grid { grid-template-columns: 1fr; }
    }
</style>
</head>
<body>
<div class="container">
<div class="header">
<div class="title">
<h1>Linux Health Dashboard</h1>
<p>Real-time Linux server monitoring</p>
</div>
<div class="status">● SYSTEM ONLINE</div>
</div>

<div class="grid">
<div class="card">
<h3>CPU Usage</h3>
<div class="value">${CPU}%</div>
<div class="progress"><div class="progress-bar" style="width:${CPU}%></div></div>
<p>${CORES} CPU cores</p>
</div>

<div class="card">
<h3>Memory Usage</h3>
<div class="value">${MEMORY}%</div>
<div class="progress"><div class="progress-bar" style="width:${MEMORY}%></div></div>
<p>Used: ${MEMORY_USED} / ${MEMORY_TOTAL}</p>
</div>

<div class="card">
<h3>Disk Usage</h3>
<div class="value">${DISK}%</div>
<div class="progress"><div class="progress-bar" style="width:${DISK}%></div></div>
<p>${DISK_USED} used / ${DISK_TOTAL}</p>
</div>

<div class="card">
<h3>Load Average</h3>

<div class="value">${LOAD}</div>
<p>Running processes: ${PROCESSES}</p>
</div>

<div class="section">
<div class="section-title">System Information</div>
<div class="info-grid">
<div class="info"><span>Hostname</span><strong>${HOSTNAME}</strong></div>

```

```

<div class="info"><span>Operating System</span><strong>${OS}</strong></div>
<div class="info"><span>Kernel</span><strong>${KERNEL}</strong></div>
<div class="info"><span>Architecture</span><strong>${ARCH}</strong></div>
<div class="info"><span>Uptime</span><strong>${UPTIME}</strong></div>
<div class="info"><span>CPU Cores</span><strong>${CORES}</strong></div>
</div>
</div>

<div class="section">
<div class="section-title">Memory Information</div>
<div class="info-grid">
<div class="info"><span>Total Memory</span><strong>${MEMORY_TOTAL}</strong></div>
<div class="info"><span>Used Memory</span><strong>${MEMORY_USED}</strong></div>
<div class="info"><span>Available Memory</span><strong>${MEMORY_AVAILABLE}</strong></div>
</div>
</div>

<div class="section">
<div class="section-title">Disk Information</div>
<div class="info-grid">
<div class="info"><span>Total Disk</span><strong>${DISK_TOTAL}</strong></div>
<div class="info"><span>Used Disk</span><strong>${DISK_USED}</strong></div>
<div class="info"><span>Free Disk</span><strong>${DISK_FREE}</strong></div>
</div>
</div>

<div class="section">
<div class="section-title">Network Information</div>
<div class="info-grid">
<div class="info"><span>Interface</span><strong>${INTERFACE}</strong></div>
<div class="info"><span>Received</span><strong>${RX}</strong></div>
<div class="info"><span>Transmitted</span><strong>${TX}</strong></div>
</div>
</div>

<div class="section">
<div class="section-title">Service Health</div>
<div class="services">
<div class="service"><div class="service-name">SSH</div><span class="badge ${SSH_CLASS}>${SSH_STATUS}</span></div>
<div class="service"><div class="service-name">Nginx</div><span class="badge ${NGINX_CLASS}>${NGINX_STATUS}</span></div>
<div class="service"><div class="service-name">Docker</div><span class="badge ${DOCKER_CLASS}>${DOCKER_STATUS}</span></div>
<div class="service"><div class="service-name">Jenkins</div><span class="badge ${JENKINS_CLASS}>${JENKINS_STATUS}</span></div>
</div>
</div>

<div class="footer">
Linux Health Dashboard<br>
Automatically refreshed every $REFRESH seconds
</div>
</div>
</body>
</html>
EOF
}

echo
echo "=====
echo "          LINUX HEALTH MONITORING DASHBOARD"
echo "=====
echo
echo "Dashboard Port : $PORT"
echo "Refresh Rate   : $REFRESH seconds"
echo "Dashboard URL   : http://SERVER-IP:$PORT"
echo
echo "Press CTRL+C to stop."
echo "=====

while true
do
    generate_html

    if command -v python3 >/dev/null 2>&1; then
        cd "$WORK_DIR" || exit 1
        exec python3 -m http.server "$PORT" --bind "$HOST"
    elif command -v busybox >/dev/null 2>&1; then
        cd "$WORK_DIR" || exit 1
        exec busybox httpd -f -p "$PORT"
    else
        echo "ERROR: Python3 or BusyBox is required only to serve the HTML dashboard."
        echo "Install Python3 on Amazon Linux: sudo dnf install python3 -y"
        exit 1
    fi
done

```

8. How Each Metric Works

CPU: Reads CPU counters from `/proc/stat` twice with a one-second interval. The difference between total CPU time and idle time is converted to a percentage.

Memory: Reads `MemTotal` and `MemAvailable` from `/proc/meminfo`. Used memory is calculated as total minus available.

Disk: Uses `df -P /` to measure the root filesystem. The dashboard displays percentage, used space, free space, and total space.

Load: Reads the first value from `/proc/loadavg`. This is the one-minute load average.

Processes: Counts numeric directories under `/proc`, which correspond to running process IDs.

Network: Detects the default route interface and reads `rx_bytes` and `tx_bytes` from `/sys/class/net`.

Services: Uses `systemctl` to determine whether `sshd`, `nginx`, `docker`, and `jenkins` are active, stopped, or not installed.

9. Testing Commands

```
# Check Bash syntax
bash -n health-dashboard.sh

# Run manually
sudo bash health-dashboard.sh

# Check the generated dashboard
ls -lh /tmp/linux-health-dashboard/index.html

# Check listening port
sudo ss -lntp | grep 8080

# Check services
sudo systemctl status sshd
sudo systemctl status docker
sudo systemctl status nginx
sudo systemctl status jenkins
```

10. Troubleshooting

Port 8080 is unreachable: Check the EC2 Security Group, local firewall, and whether the server is listening with `ss -lntp`.

Python3 not found: On Amazon Linux 2023, install it with `sudo dnf install python3 -y`. On Ubuntu, use `sudo apt update && sudo apt install python3 -y`.

Service shows NOT INSTALLED: That service is not registered on the machine. This is normal if Nginx, Docker, or Jenkins has not been installed.

Permission denied: Run the dashboard with `sudo`. Do not make the entire filesystem writable just to solve a permission problem.

CPU appears slow to load: CPU collection intentionally waits one second between `/proc/stat` samples to calculate utilization.

11. Stop the Dashboard

```
Press CTRL+C in the terminal running the script.

# If you started it in the background, find it:
ps aux | grep health-dashboard

# Then terminate the relevant process:
sudo kill <PID>
```

12. Optional Background Execution

```
sudo nohup ./health-dashboard.sh > dashboard.log 2>&1 &
```

```
# Follow the log
sudo tail -f dashboard.log
```

13. Resume / Interview Explanation

A good explanation is: *"I built a Bash-based Linux health monitoring dashboard. The script collects CPU, memory, disk, load, process, network, and systemd service metrics directly from Linux interfaces such as /proc, /sys, df, free, and systemctl. It generates a responsive HTML dashboard and exposes it through a lightweight HTTP server on port 8080. This project helped me practice Linux administration, Bash scripting, monitoring, troubleshooting, and server deployment."*

14. Recommended Future Improvements

- Add CPU temperature where supported
- Add per-process CPU and memory tables
- Add configurable warning/critical thresholds
- Add log monitoring from /var/log
- Add email/SNS alerts
- Add systemd service for automatic startup
- Add historical metric storage and graphs
- Containerize the dashboard if you later want a Docker version

15. Project Structure

```
linux-health-dashboard/
■■■■ health-dashboard.sh

Runtime files:
■■■■ /tmp/linux-health-dashboard/
■■■■ index.html
```